



Data Analytics Project: House Sales Analysis

Team1

- 1 Gayathri Dwarapudi
- 2 Manteena Vineeth
- 3 Vivek vardhan
- 4 Suddula Yashwanth

Dataset Used: kc_house_data.csv

This project analyzes house sales data to understand how different factors affect house prices. The dataset includes details like price, size, number of bedrooms, and location. Using data cleaning, analysis, and visualization, we find useful patterns and trends. The main goal is to identify what influences house prices the most.

Project Goal: The goal of this project is to analyze house sales data and understand the factors that affect house prices. It aims to identify important features such as size, location, and number of rooms that influence pricing. The project also helps in finding patterns and trends in the data using analysis and visualization techniques.

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts)

Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Data Cleaning, Type Conversion.
Pandas Operations	Column Selection, Filtering, Grouping, Aggregation, Value Count, Feature Engineering (Height Category creation)
NumPy Calculations	Array conversion, Numerical analysis using (np.diff) for height differences
Visualization	Trend Analysis, Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning Description:

In this Scenario,

The House Sales dataset (kc_house_data.csv) is loaded using the Pandas library. The first 5 rows and column names are displayed to understand the dataset structure. Missing values are checked in important columns like bedrooms, bathrooms, sqft_living, and price.

Missing values are filled using mode for bedrooms and mean for bathrooms, sqft_living, and price.

Finally, the columns are converted to numeric format and checked again to ensure the data is clean.

Task	Description
1.	Load the <code>kc_house_data</code> dataset using Pandas.
2.	Display the first 5 rows (<code>head()</code>) and Column names
3.	Check for missing values in: bedrooms, bathrooms, sqft_living & price
4.	Handling missing values: Fill missing values: bedrooms → use mode bathrooms → use mean sqft_living → use mean price → use mean
5.	Convert these columns to numeric if required: bedrooms, bathrooms, sqft_living & price

Results of Scenario 1:

```
First 5 Rows:
   id      date      price  ...      long  sqft_living15  sqft_lot15
0  7129300520  20141013T0000000  221900.0  ... -122.257      1340      5650
1  6414100192  20141209T0000000  538000.0  ... -122.319      1690      7639
2  5631500400  20150225T0000000  180000.0  ... -122.233      2720      8062
3  2487200875  20141209T0000000  604000.0  ... -122.393      1360      5000
4  1954400510  20150218T0000000  510000.0  ... -122.045      1800      7503

[5 rows x 21 columns]
```

```
Column Names:
Index(['id', 'date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
      'sqft_lot', 'floors', 'waterfront', 'view', 'condition', 'grade',
      'sqft_above', 'sqft_basement', 'yr_built', 'yr_renovated', 'zipcode',
      'lat', 'long', 'sqft_living15', 'sqft_lot15'],
      dtype='str')
```

```
Missing Values Before Filling:
bedrooms: 0
bathrooms: 0
sqft_living: 0
price: 0
```

```
Missing Values After Filling:
bedrooms: 0
bathrooms: 0
sqft_living: 0
price: 0
```

Conclusion:

This scenario helps in preparing the dataset for further analysis by performing basic cleaning operations. Missing values were identified and replaced using suitable statistical methods like mode and mean. Converting columns into numeric format improves data accuracy and prevents errors in future calculations.

As a result, the dataset becomes clean, consistent, and ready for advanced data analysis such as price prediction, visualization, and trend analysis in house sales.

Scenario 2:

Description:

Line Graph (Price Trend) (□ Easy)

Line graph visualizes the price fluctuations across the first ten entries of the dataset. By isolating the id and price columns and converting the price values into a NumPy array, the data was prepared for a clean plotting process. The X-axis represents the sequential index of the records, ranging from 0 to 9, while the Y-axis tracks the corresponding house prices. The resulting plot clearly labels the trend with a descriptive title and axis titles, ensuring the visual information is easy to interpret.

Task	Description
1.	Select: id & price
2.	Take first 10 rows only.
3.	Convert the Price into a NumPy Array
4.	Plot a line graph with X-axis – (index [0-9]), Y-axis – (Height)
5.	Add the title and labels
6.	Save the graph: <code>plt.savefig("house_prices_line.png")</code>

Graph :



Conclusion:

The visualization provides a focused look at the initial pricing trends within the data. By strictly limiting the scope to the first 10 rows, we can identify specific volatility or patterns that might be lost in a larger dataset. The process of converting the data to a

Scenario 3:

Description:

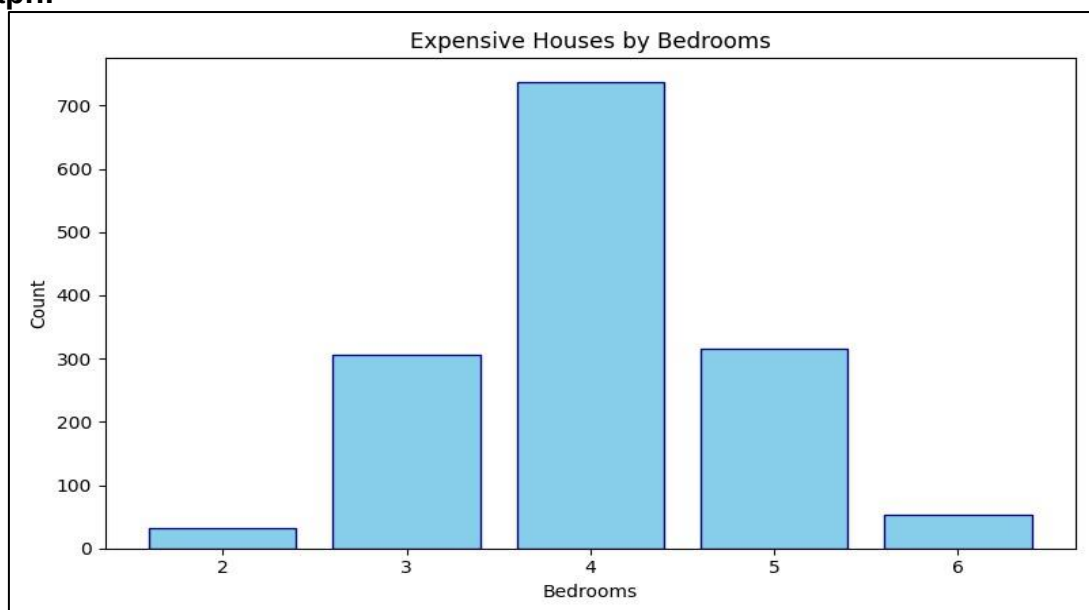
NumPy array and using Matplotlib's saving function ensures that the insights are not only viewable but also preserved as a high-quality image file.

Filtering + Bar Chart (□ Medium)

Here we are going to Filter the houses which are above the threshold value and get to know which are expensive houses and which are near to the threshold value. The data is then grouped by Bedrooms to study distribution patterns. A Bar chart is created and saved to visualize the plots.

Task	Description
1.	Filter houses where: ○ price > 1000000
2.	Count number of houses per: ○ bedrooms
3.	Select top bedroom categories.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: ○ X-axis → Bedrooms ○ Y-axis → Count
6.	Rotate labels if needed.
7.	Save graph: plt.savefig("expensive_houses_bar.png")

Graph:



Conclusion:

Scenario 4:

Description:

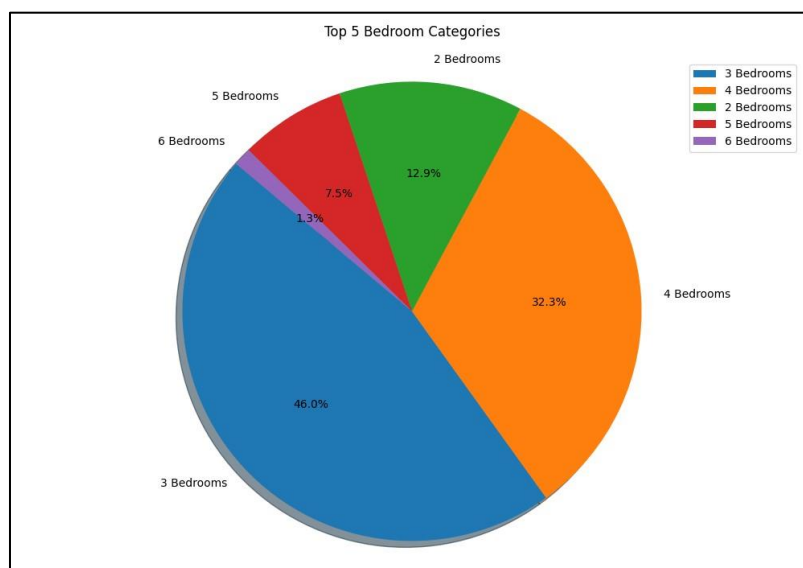
From the above chart, we can analyse that the house prices vary with the number of bedrooms and certain bedrooms categories are more common among expensive houses. Here we can see that 4 bedroom houses are more expensive than the 6 and 2 bedrooms which means, 4 bedroom houses have a high demand and many would prefer them so it costed higher than 6 bedroom flats.

Pie Chart (Region Distribution) + Save (□ Medium)

Here we are going to build Region distribution pie chart in order to understand the overall distribution of top 5 bedrooms And we will construct a frequency of each house type based on bedrooms column to count number of houses. Labels and values are prepared in order to showcase in pie chart.

Task	Description
1.	Count number of houses by: ○ bedrooms
2.	Select top 5 bedrooms category.
3.	Prepare labels and values.
4.	Plot a pie chart.
5.	Add percentage labels using <code>autopct</code> (e.g., '%.1f%%').
6.	Save the graph: <code>plt.savefig("bedroom_distribution.png")</code>

Graph:



Conclusion:

Scenario 5:

Description:

From the above pie chart, we can understand that the distribution of house types by visualizing the most common bedroom configuration. By filtering for top 5 categories, the analysis provides a concise overview of the market, highlighting 3 and 4 bedroom houses make up the majority of the inventory. The use of percentage labels and a pie chart format makes the data easy to interpret at a glance. The lowest demanded houses are considered as 6 bedroom houses where it just had 1.3% of the total distribution.

Advanced Analysis + Multiple Graphs (🟡 Hard)

Here we perform feature engineering steps in order to analyse the trends deeply. A new column Price Category is created to classify houses into Luxury, Mid range, and Affordable based on price. The price column is converted into a NumPy array. All graphs are saved for reporting purposes.

Task	Description
1.	Feature Creation Create a new column: Price Category • price $\geq$ 1000000 $\rightarrow$ "Luxury" • 500000 – 999999 $\rightarrow$ "Mid Range" • $<$ 500000 $\rightarrow$ "Affordable"
2.	1. Convert price column to a NumPy array. 2. Calculate price differences using: np.diff()
3.	Visualizations
4.	Save all graphs plt.savefig("price_trend.png") plt.savefig("price_category_stacked.png") plt.savefig("price_histogram.png")
5.	Save the graphs: <code>plt.savefig("price_trend.png"), plt.savefig("price_category_stacked.png"), plt.savefig("price_histogram.png").</code>

Scenario 6:

Description:

6.

Identify:

1. Which bedroom category has the most expensive houses?
2. Which price category is most common?
3. What is the distribution pattern of house prices?
 - Right-skewed?
 - Normal?
 - Concentrated in a lower price range?

Graph:

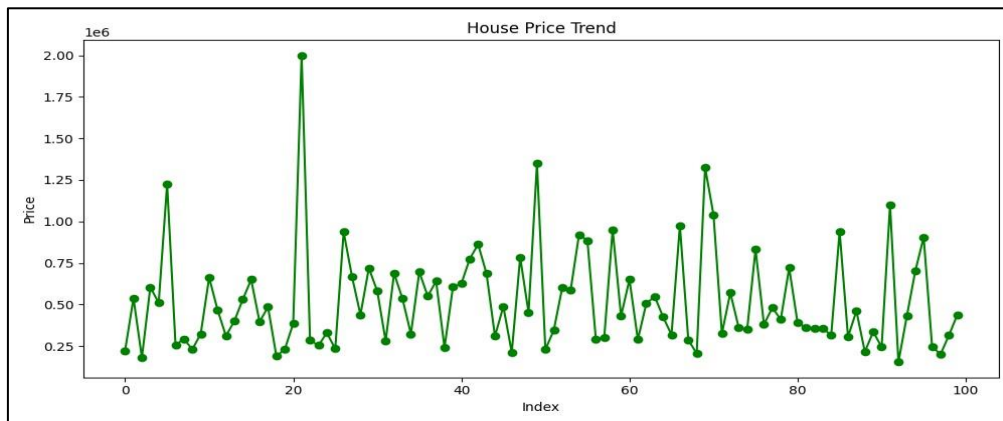


Fig 5.1 House Price Trend

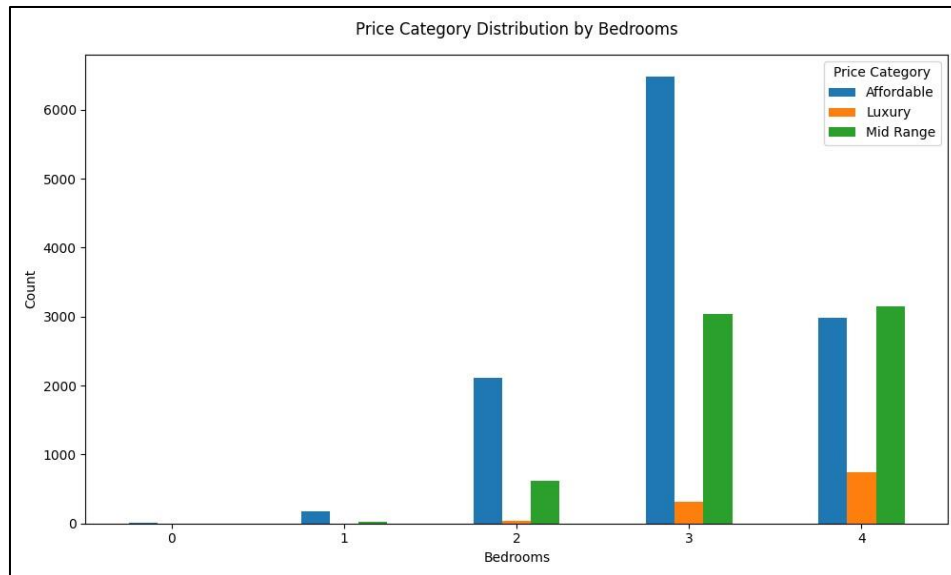


Fig 5.2 Price Category Distribution by Bedrooms



Fig 5.3 Price Distribution

Conclusion:

The analysis shows that houses with more bedrooms are more likely to fall under the luxury category. Most houses belong to the affordable or mid-range categories, indicating fewer high-priced properties. The price distribution is right-skewed, meaning most values are concentrated in the lower range with a few extreme high prices.

- The Housing market in this dataset is dominated by moderately priced homes with a few expensive properties creating a right – skewed distribution.
- The house prices exhibit high variability with no consistent trend, dominated by mid – range values and influenced by a few high – value outliers. This indicates a heterogenous housing market with both affordable and luxury segments.
- Overall, the dataset reflects a typical real estate pattern with uneven price distribution.

Final Conclusion:

This project helped analyze the KC House Sales dataset using Python libraries such as Pandas, NumPy, and Matplotlib. The dataset was cleaned by handling missing values and converting important columns into proper numeric formats. Different visualizations like line graphs, bar charts, pie charts, stacked bar charts, and histograms were created to understand house prices and bedroom distribution. Expensive houses were filtered to study luxury homes, and price categories were created for deeper analysis. The results showed that house prices are mostly concentrated in lower and mid ranges, while luxury houses are fewer in number.

What we have Learnt?

We learnt how to load and clean real-world datasets using Pandas. We understood the importance of handling missing values using mean and mode methods and converting columns into proper numeric data types. We practiced filtering data based on specific conditions and grouping data using functions like `groupby()` and `value_counts()`. We also learnt how to use NumPy arrays and functions such as `np.diff()` for analysis. Using Matplotlib, we created different types of graphs and saved them using `plt.savefig()`. Most importantly, we learnt how to identify patterns in data and draw meaningful conclusions from the analysis.